

Physique dans le jeu vidéo

Table des matières

Session 1 : Géométrie et Calcul Vectoriel	3
Session 2 : Cinématique 2D et Mouvement de Projectile	10
Position en 2D	11
Vitesse en 2D	12
Accélération en 2D	14
Session 3 : Les lois de Newton – La méthode physique	17
Objectifs de la session	17
La démarche physique : modéliser l'univers	17
Session 4 : Les forces de la nature	19
Objectifs de la session	19
Session 5 : Intégration numérique 101 - Méthode d'Euler	21
Objectifs de la session	21
Session 6 : L'impulsion, les collisions	23
Objectifs de la session	23
Session 7 : Broad et Narrow phases – Part 1	25
Objectifs de la session	25
Session 8 : Les ressorts - Méthode de Verlet	27
Objectifs de la session	27
Session 9 : TP - Particules	29
Objectifs du TP	29
Session 10 : Introduction à la physique des rotations	31
Objectifs de la session	31
Étape 1 : Classe RigidBody	32
Étape 2 : ApplyForce(force, worldPoint)	32
Étape 3 : Intégrateur	32
Session 11 : Broad et Narrow phases – Part 2	33
Objectifs de la session	33
Session 12 : TP - Les planètes - Runge-Kutta	35
Objectifs du TP	35
Session 13 : Les moteurs physiques commerciaux	37
Objectifs de la session	37
Session 14 : Corps rigides – Boite de collision	39
Objectifs de la session	39
Session 15 : Joints, Moteurs	41
Objectifs de la session	41
Session 16 : Les contraintes – et l'IK	43
Objectifs de la session	43
Session 17 : TP - Véhicules	46
Objectifs du TP	46
Session 18 : Personnages	48
Objectifs de la session	48
Session 19 : Physique, GPU et IA	50

Objectifs de la session	50
Session 20 : Résumé - Quiz final	52
Objectifs de la session	52
Concepts clés à retenir	52
Quiz final	52

Session 1 : Géométrie et Calcul Vectoriel

Définitions

Un vecteur est une grandeur ayant une direction et une magnitude.

On dit qu'il est de dimension n si il a n composantes.

Dans le plan, $n = 2$, donc un vecteur a 2 composantes, 2 dimensions.

🔗 Représentation

Un vecteur est représenté par une flèche. Il peut être placé n'importe où dans le plan. On le place habituellement par rapport au contexte de ce qu'il représente.



Fig. 1. – Un vecteur

💡 Notation vectorielle

Même si le vecteur est défini par ses magnitude et direction, on utilise une notation vectorielle, qui décompose le vecteur selon les vecteurs $\hat{i}$ et $\hat{j}$ du plan cartésien. On retrouve ses 2 dimensions représentées en colonne.

Exemple.

$$\vec{v} = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = 2\hat{i} + 3\hat{j}$$

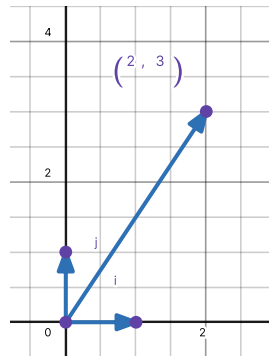


Fig. 2. – Le vecteur $\vec{v}$

🔗 Déplacement

Un déplacement peut se décrire par le vecteur reliant le point A (départ) au point B (arrivée) :

$$\overrightarrow{AB} = \begin{pmatrix} x_B - x_A \\ y_B - y_A \end{pmatrix}$$

🔗 Des vecteurs célèbres en physique

- **Le vecteur position**

Souvent noté $\vec{r}$ (radius), représente la position d'un point dans l'espace en partant de l'origine.

- **Le vecteur vitesse**

Noté $\vec{v}$, représente la vitesse d'un objet, souvent représenté partant du centre de l'objet.

- **Le vecteur accélération**

Noté $\vec{a}$, représente l'accélération d'un objet, souvent représenté partant du centre de l'objet.

Magnitude et direction

La longueur du vecteur, aussi appelée magnitude, est notée $\|\vec{v}\|$, et parfois simplement $|\vec{v}|$.

En 2D : On utilise le théorème de Pythagore pour calculer la norme (autre nom de la magnitude) :

$$\|\vec{v}\| = \sqrt{v_x^2 + v_y^2}$$

Vecteur Unitaire ($\hat{u}$) : Vecteur de longueur 1 direction $\vec{v}$:

$$\hat{u} = \frac{\vec{v}}{\|\vec{v}\|}$$

Un vecteur unitaire est un vecteur de longueur 1, il est souvent utilisé pour représenter une direction sans avoir à se soucier de la magnitude.

Opérations de Base

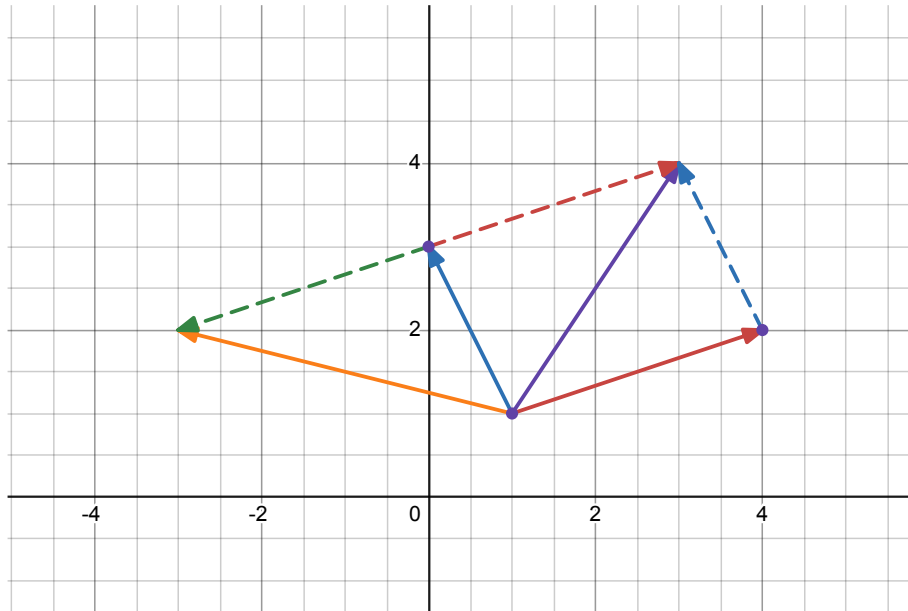
Soit $\vec{u} = \begin{pmatrix} u_x \\ u_y \end{pmatrix}$ et $\vec{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$.

Addition :

$$\vec{u} + \vec{v} = \begin{pmatrix} u_x + v_x \\ u_y + v_y \end{pmatrix}$$

Soustraction :

$$\vec{u} - \vec{v} = \begin{pmatrix} u_x - v_x \\ u_y - v_y \end{pmatrix}$$

Fig. 3. – Le vecteur $\vec{u}$ (rouge) et le vecteur $\vec{v}$ (bleu)

Multiplication par scalaire (k) :

$$k \cdot \vec{u} = \begin{pmatrix} ku_x \\ ku_y \end{pmatrix}$$

Produit Scalaire

Résultat : un **nombre** (scalaire).

Formule algébrique :

$$\vec{u} \cdot \vec{v} = u_x v_x + u_y v_y + u_z v_z$$

Formule géométrique :

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \cdot \|\vec{v}\| \cdot \cos(\theta)$$

1. Si $\vec{u} \cdot \vec{v} = 0$, alors $\vec{u} \perp \vec{v}$.
2. Si $\vec{u} \cdot \vec{v} > 0$, alors l'angle entre les vecteurs est inférieur à 90° . (devant)
3. Si $\vec{u} \cdot \vec{v} < 0$, alors l'angle entre les vecteurs est supérieur à 90° . (derrière)

Angle entre 2 vecteurs

On isole $\cos(\theta)$. :

$$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}$$

$$\theta = \arccos\left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}\right)$$

Projections

Projection de $\vec{u}$ sur $\vec{v}$. On utilise le produit scalaire, dans la direction sur laquelle on projette.

$$\text{proj}_{\vec{v}}(\vec{u}) = \left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{v}\|^2} \right) \cdot \vec{v}$$

Et si $\vec{v}$ est normalisé (direction de la droite sur laquelle on projète), on peut simplifier :

$$\text{proj}_{\vec{v}}(\vec{u}) = (\vec{u} \cdot \vec{v}) \cdot \vec{v}$$

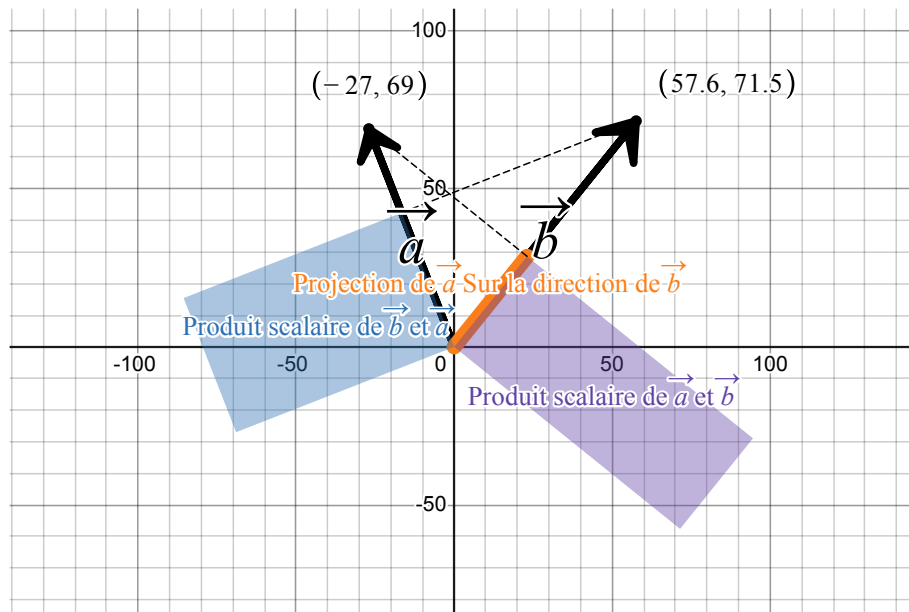


Fig. 4. – Produit scalaire, projection

Produit Vectoriel (3D)

Résultat : un **vecteur** perpendiculaire.

$$\vec{w} = \vec{a} \times \vec{b}$$

Commutativité : Attention, le produit vectoriel n'est pas commutatif ! On a un changement de signe :

$$\vec{a} \times \vec{b} = -\vec{b} \times \vec{a}$$

Magnitude :

$$\|\vec{a} \times \vec{b}\| = \|\vec{a}\| \cdot \|\vec{b}\| \cdot \sin(\theta)$$

Direction : Le produit vectoriel est perpendiculaire aux deux vecteurs :

$$\vec{a} \times \vec{b} \perp \vec{a}$$

$$\vec{a} \times \vec{b} \perp \vec{b}$$

On utilise la règle de la main droite pour trouver la direction du produit vectoriel.

- Le pouce pointe sur a

- l'index sur b
- le majeur sur $a \times b$.

Calcul (Déterminant) : Le déterminant est un outil algébrique qui permet de trouver les solutions d'un problème de n équations avec n inconnues. Cette solution représente, en quelque sorte une direction émergente (perpendiculaire) des équations.

$$a = \begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} \text{ et } b = \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix}$$

$$\vec{a} \times \vec{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

Exercice : Le parallélogramme

Soit deux vecteurs dans l'espace 3D :

$$\vec{a} = \begin{pmatrix} 2 \\ 1 \\ 0 \end{pmatrix} \text{ et } \vec{b} = \begin{pmatrix} 0 \\ 3 \\ 2 \end{pmatrix}$$

1. Calculez le produit vectoriel $\vec{w} = \vec{a} \times \vec{b}$ en utilisant la méthode du déterminant.
2. Vérifiez que $\vec{w}$ est bien perpendiculaire à $\vec{a}$ et à $\vec{b}$ à l'aide du produit scalaire.
3. Calculez l'aire du parallélogramme formé par $\vec{a}$ et $\vec{b}$.

Solution : Produit vectoriel par le déterminant. On pose la matrice avec $\hat{i}, \hat{j}, \hat{k}$ en première ligne :

$$\vec{a} \times \vec{b} = \begin{pmatrix} \hat{i} & \hat{j} & \hat{k} \\ 2 & 1 & 0 \\ 0 & 3 & 2 \end{pmatrix}$$

Développement par la première ligne (cofacteurs) :

$$\vec{w} = \hat{i} \cdot \begin{pmatrix} 1 & 0 \\ 3 & 2 \end{pmatrix} - \hat{j} \cdot \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} + \hat{k} \cdot \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$$

Calcul des sous-déterminants 2x2 :

$$\vec{w} = \hat{i} \cdot (1 \cdot 2 - 0 \cdot 3) - \hat{j} \cdot (2 \cdot 2 - 0 \cdot 0) + \hat{k} \cdot (2 \cdot 3 - 1 \cdot 0)$$

$$\vec{w} = \hat{i} \cdot (2) - \hat{j} \cdot (4) + \hat{k} \cdot (6)$$

$$\vec{w} = \begin{pmatrix} 2 \\ -4 \\ 6 \end{pmatrix}$$

Solution : Vérification de perpendicularité.

$$\vec{w} \cdot \vec{a} = (2)(2) + (-4)(1) + (6)(0) = 4 - 4 + 0 = 0 \quad \text{checkmark}$$

$$\vec{w} \cdot \vec{b} = (2)(0) + (-4)(3) + (6)(2) = 0 - 12 + 12 = 0 \quad \text{checkmark}$$

Les deux produits scalaires sont nuls : $\vec{w}$ est bien perpendiculaire à $\vec{a}$ et $\vec{b}$.

Solution : Aire du parallélogramme. L'aire du parallélogramme formé par deux vecteurs est égale à la norme de leur produit vectoriel :

$$\text{Aire} = \|\vec{w}\| = \sqrt{2^2 + (-4)^2 + 6^2} = \sqrt{4 + 16 + 36} = \sqrt{56} \approx 7.48$$

Exercice : Le food truck

Un food truck vend trois types de repas : des burgers à 8, des tacos à 5, et des salades à 6.

Samedi, il a vendu 100 repas pour un total de 620 dollars.

On sait de plus que le nombre de tacos vendus est égal au nombre de burgers plus le double du nombre de salades.

Combien de chaque repas a-t-il vendu ?

Mise en équation :

Soit x le nombre de burgers, y le nombre de tacos, z le nombre de salades.

1. « Total des repas » : $x + y + z = 100$
2. « Total des ventes » : $8x + 5y + 6z = 620$
3. « Relation tacos » : $y = x + 2z$, soit $-x + y - 2z = 0$

Système :

$$x + y + z = 100$$

$$8x + 5y + 6z = 620$$

$$-x + y - 2z = 0$$

Solution : Règle de Cramer. Si $\Delta \neq 0$, alors $x = \frac{\Delta_x}{\Delta}$, $y = \frac{\Delta_y}{\Delta}$, $z = \frac{\Delta_z}{\Delta}$.

Déterminant principal Δ :

$$\Delta = \begin{pmatrix} 1 & 1 & 1 \\ 8 & 5 & 6 \\ -1 & 1 & -2 \end{pmatrix}$$

Développement par la première ligne :

$$\Delta = 1 \cdot \begin{pmatrix} 5 & 6 \\ 1 & -2 \end{pmatrix} - 1 \cdot \begin{pmatrix} 8 & 6 \\ -1 & -2 \end{pmatrix} + 1 \cdot \begin{pmatrix} 8 & 5 \\ -1 & 1 \end{pmatrix}$$

$$\Delta = 1 \cdot (-10 - 6) - 1 \cdot (-16 + 6) + 1 \cdot (8 + 5)$$

$$\Delta = -16 - (-10) + 13 = -16 + 10 + 13 = 7$$

Solution : Calcul de Δ_x , Δ_y , Δ_z . Δ_x (remplacer la colonne 1 par les constantes) :

$$\Delta_x = \begin{pmatrix} 100 & 1 & 1 \\ 620 & 5 & 6 \\ 0 & 1 & -2 \end{pmatrix}$$

$$\Delta_x = 100 \cdot (-16) - 1 \cdot (-1240) + 1 \cdot (620) = -1600 + 1240 + 620 = 260$$

Δ_y (remplacer la colonne 2 par les constantes) :

$$\Delta_y = \begin{pmatrix} 1 & 100 & 1 \\ 8 & 620 & 6 \\ -1 & 0 & -2 \end{pmatrix}$$

$$\Delta_y = 1 \cdot (-1240) - 100 \cdot (-10) + 1 \cdot (620) = -1240 + 1000 + 620 = 380$$

Δ_z (remplacer la colonne 3 par les constantes) :

$$\Delta_z = \begin{pmatrix} 1 & 1 & 100 \\ 8 & 5 & 620 \\ -1 & 1 & 0 \end{pmatrix}$$

$$\Delta_z = 1 \cdot (-620) - 1 \cdot (620) + 100 \cdot (13) = -620 - 620 + 1300 = 60$$

Solution : Résultats et vérification.

$$x = \frac{\Delta_x}{\Delta} = \frac{260}{7} \approx 37 \text{ burgers}$$

$$y = \frac{\Delta_y}{\Delta} = \frac{380}{7} \approx 54 \text{ tacos}$$

$$z = \frac{\Delta_z}{\Delta} = \frac{60}{7} \approx 9 \text{ salades}$$

Vérification :

1. $37 + 54 + 9 = 100$ checkmark
2. $8(37) + 5(54) + 6(9) = 296 + 270 + 54 = 620$ checkmark
3. $54 = 37 + 2(9) = 37 + 18 = 55 \approx 54$ checkmark (arrondi)

Note

Les valeurs exactes sont $x = \frac{260}{7}$, $y = \frac{380}{7}$, $z = \frac{60}{7}$. Dans un cas réel, on ajusterait les données pour obtenir des nombres entiers. L'important ici est la méthode.

Session 2 : Cinématique 2D et Mouvement de Projectile

🔗 Introduction à la cinématique

La cinématique est la branche de la mécanique qui décrit le mouvement des objets sans considérer les causes du mouvement (les forces). Contrairement à la dynamique, qui relie le mouvement aux forces.

Notre objectif dans ce bloc est de développer les outils mathématiques pour décrire précisément *comment* les objets se déplacent.

Position, Vitesse et Accélération

La position, la vitesse et l'accélération sont des concepts qui permettent de décrire précisément comment un objet se comporte.

- La **position** est le vecteur qui indique la localisation d'un point dans l'espace.
- La **vitesse** est le vecteur qui indique la vitesse du point, définie comme la variation (dérivée) de la position par rapport au temps.
- L'**accélération** est le vecteur qui indique l'accélération du point, définie comme la variation (dérivée) de la vitesse par rapport au temps.

🔗 Cas particuliers

- Lorsque la position est constante, le point est au repos, immobile. Sa vitesse et son accélération sont nulles.
- Lorsque la vitesse est constante, le point se déplace à vitesse constante en ligne droite. Son accélération est nulle.
- Lorsque l'accélération est constante, la vitesse augmente constamment, le point se déplace de plus en plus vite.

1. Introduction et Rappels

Rappels sur les vecteurs

Rappel de la définition d'un vecteur comme une quantité possédant une magnitude (longueur, norme) et une direction.

Représentation d'un vecteur en 2D à l'aide de ses composantes :

$$\vec{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

.

Opérations vectorielles de base :

$$\vec{a} + \vec{b} = \begin{pmatrix} a_x + b_x \\ a_y + b_y \end{pmatrix}$$

$$\vec{a} - \vec{b} = \begin{pmatrix} a_x - b_x \\ a_y - b_y \end{pmatrix}$$

Multiplication par un scalaire :

$$k\vec{a} = \begin{pmatrix} ka_x \\ ka_y \end{pmatrix}$$

Position en 2D

Vecteur Position

Pour décrire la localisation d'un point dans un plan bidimensionnel, nous utilisons le **vecteur position**, noté $\vec{r}$ (ou parfois $\vec{s}$ ou $\vec{x}$).

Avec un système de coordonnées cartésiennes (x, y) , le vecteur position d'un point P de coordonnées (x, y) est :

$$\vec{r} = \begin{pmatrix} x \\ y \end{pmatrix} = x\hat{i} + y\hat{j}$$

où $\hat{i} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ et $\hat{j} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ sont les vecteurs unitaires.

- Le vecteur position pointe de l'origine vers la position de l'objet.
- La **magnitude** $|\vec{r}| = \sqrt{x^2 + y^2}$ représente la distance à l'origine.
- La **direction** est donnée par l'angle θ avec $\tan(\theta) = \frac{y}{x}$.

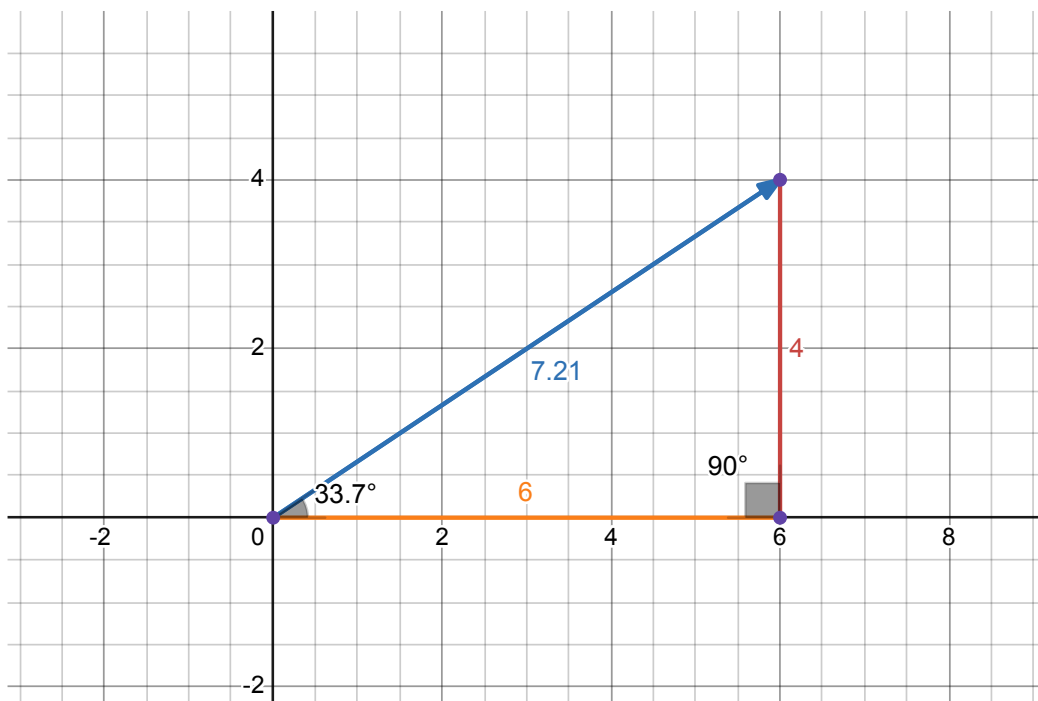


Fig. 5. – Plan 2D avec vecteurs position et vitesse

💡 Trajectoire

Si la position d'un objet change au cours du temps, on décrit son mouvement par :

$$\vec{r}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$$

L'ensemble des points atteints forme la **trajectoire** — une courbe dans l'espace. $x(t)$ et $y(t)$ sont des fonctions du temps.

Exemples de trajectoires :

• **Mouvement rectiligne uniforme :**

$$\vec{r}(t) = \begin{pmatrix} x_0 + v_{0x}t \\ y_0 + v_{0y}t \end{pmatrix}$$

• **Mouvement circulaire uniforme :**

$$\vec{r}(t) = \begin{pmatrix} R \cos(\omega t) \\ R \sin(\omega t) \end{pmatrix}$$

• **Mouvement parabolique (projectile) :**

$$\vec{r}(t) = \begin{pmatrix} x_0 + v_{0x}t \\ y_0 + v_{0y}t - \frac{1}{2}gt^2 \end{pmatrix}$$

Vitesse en 2D

Vitesse Moyenne

Le **déplacement** d'un objet pendant un intervalle $\Delta t = t_f - t_i$ est :

$$\Delta \vec{r} = \vec{r}_f - \vec{r}_i = \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}$$

Le **vecteur vitesse moyenne** $\vec{v}_{\text{moy}}$ est le rapport du déplacement au temps écoulé :

$$\vec{v}_{\text{moy}} = \frac{\Delta \vec{r}}{\Delta t} = \begin{pmatrix} \frac{\Delta x}{\Delta t} \\ \frac{\Delta y}{\Delta t} \end{pmatrix} = \begin{pmatrix} v_{\text{moy},x} \\ v_{\text{moy},y} \end{pmatrix}$$

La vitesse moyenne a la même direction que le déplacement, et sa magnitude est le déplacement total divisé par le temps écoulé.

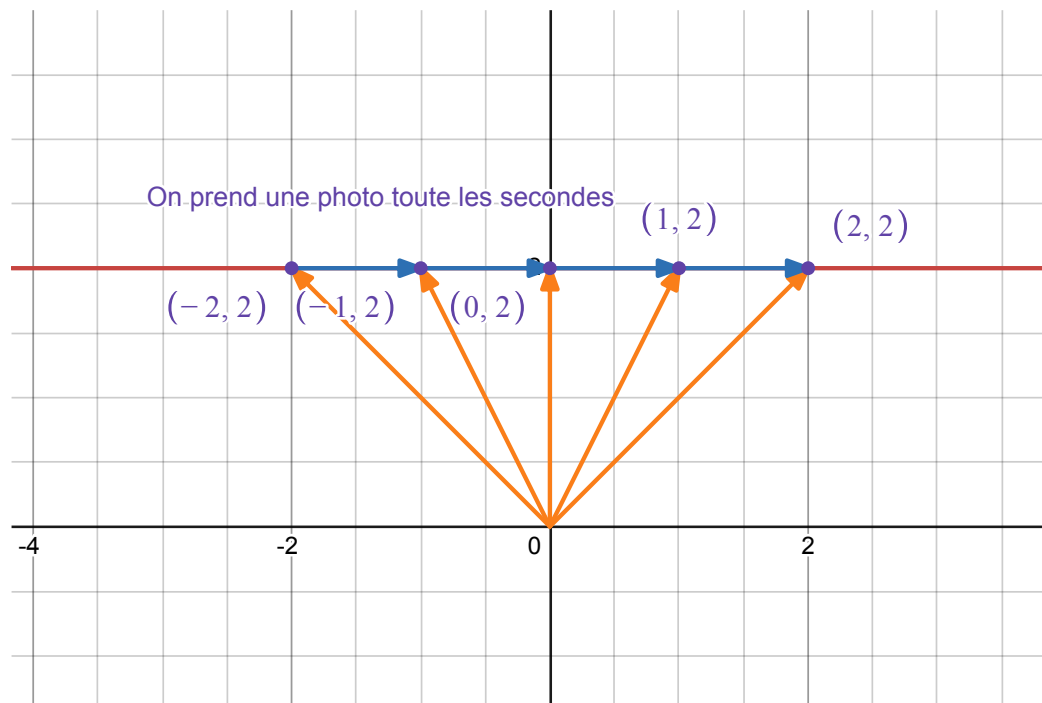


Fig. 6. – Plan 2D avec vecteurs position et vitesse

Exemple. Si une voiture se déplace de $\vec{r}_i = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ à $\vec{r}_f = \begin{pmatrix} 10 \\ 5 \end{pmatrix}$ en $\Delta t = 5$ secondes, alors le déplacement est $\Delta \vec{r} = \begin{pmatrix} 10 \\ 5 \end{pmatrix}$ et la vitesse moyenne est $\vec{v}_{\text{moy}} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ m/s.

Vitesse Instantanée

La **vitesse instantanée** $\vec{v}(t)$ est la limite de la vitesse moyenne lorsque $\Delta t \rightarrow 0$:

$$\vec{v}(t) = \lim_{\Delta t \rightarrow 0} \frac{\Delta \vec{r}}{\Delta t} = \frac{d\vec{r}}{dt}$$

En composantes :

$$\vec{v}(t) = \begin{pmatrix} \frac{dx(t)}{dt} \\ \frac{dy(t)}{dt} \end{pmatrix} = \begin{pmatrix} v_x(t) \\ v_y(t) \end{pmatrix}$$

- La **magnitude** $|\vec{v}(t)| = \sqrt{v_{x(t)}^2 + v_{y(t)}^2}$ est la **vitesse scalaire**.
- La **direction** est tangente à la trajectoire au point considéré.

🔗 Application aux types de trajectoire

1. Mouvement rectiligne uniforme :

$$\vec{v}(t) = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

— Le vecteur vitesse est **constant**.

2. Mouvement circulaire uniforme :

$$\vec{v}(t) = \begin{pmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \end{pmatrix}$$

Magnitude constante : $|\vec{v}| = R\omega$. La vitesse est tangente au cercle.

3. Mouvement parabolique (projectile) :

$$\vec{v}(t) = \begin{pmatrix} v_{0x} \\ v_{0y} - gt \end{pmatrix}$$

La composante horizontale v_x reste constante, la verticale v_y diminue linéairement.

Accélération en 2D

Accélération Moyenne

Le **vecteur accélération moyenne** $\vec{a}_{\text{moy}}$ pendant un intervalle $\Delta t = t_f - t_i$ est le rapport du changement de vitesse au temps écoulé :

$$\vec{a}_{\text{moy}} = \frac{\Delta \vec{v}}{\Delta t} = \begin{pmatrix} \frac{\Delta v_x}{\Delta t} \\ \frac{\Delta v_y}{\Delta t} \end{pmatrix} = \begin{pmatrix} a_{\text{moy},x} \\ a_{\text{moy},y} \end{pmatrix}$$

L'accélération moyenne a la direction du changement de vitesse.

Accélération Instantanée

L'**accélération instantanée** $\vec{a}(t)$ est la dérivée de la vitesse par rapport au temps :

$$\vec{a}(t) = \frac{d\vec{v}}{dt} = \begin{pmatrix} \frac{dv_x(t)}{dt} \\ \frac{dv_y(t)}{dt} \end{pmatrix} = \begin{pmatrix} \frac{d^2x(t)}{dt^2} \\ \frac{d^2y(t)}{dt^2} \end{pmatrix}$$

L'accélération peut changer la magnitude de la vitesse (accélérer/décélérer), sa direction, ou les deux.

Cas particulier : Accélération constante

Si l'accélération $\vec{a}$ est constante, on peut **intégrer** pour obtenir la vitesse et la position :

$$\vec{v}(t) = \vec{v}_0 + \vec{a}t = \begin{pmatrix} v_{0x} + a_x t \\ v_{0y} + a_y t \end{pmatrix}$$

$$\vec{r}(t) = \vec{r}_0 + \vec{v}_0 t + \frac{1}{2} \vec{a} t^2 = \begin{pmatrix} x_0 + v_{0x} t + \frac{1}{2} a_x t^2 \\ y_0 + v_{0y} t + \frac{1}{2} a_y t^2 \end{pmatrix}$$

où $\vec{v}_0 = \begin{pmatrix} v_{0x} \\ v_{0y} \end{pmatrix}$ est la vitesse initiale et $\vec{r}_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$ est la position initiale.

C'est ce cas particulier qui est crucial pour l'étude du mouvement de projectile sous l'effet de la gravité.

🔗 Exemples par type de trajectoire

1. Mouvement rectiligne uniforme :

- Position : $\vec{r}(t) = \begin{pmatrix} x_0 + v_x t \\ y_0 + v_y t \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

La dérivée d'un vecteur constant est nulle. Aucune accélération n'est nécessaire pour maintenir le mouvement rectiligne uniforme.

2. Mouvement circulaire uniforme :

- Position : $\vec{r}(t) = \begin{pmatrix} R \cos(\omega t) \\ R \sin(\omega t) \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} -R\omega^2 \cos(\omega t) \\ -R\omega^2 \sin(\omega t) \end{pmatrix}$

On observe que $\vec{a}(t) = -\omega^2 \vec{r}(t)$: l'accélération est dirigée vers le centre du cercle.

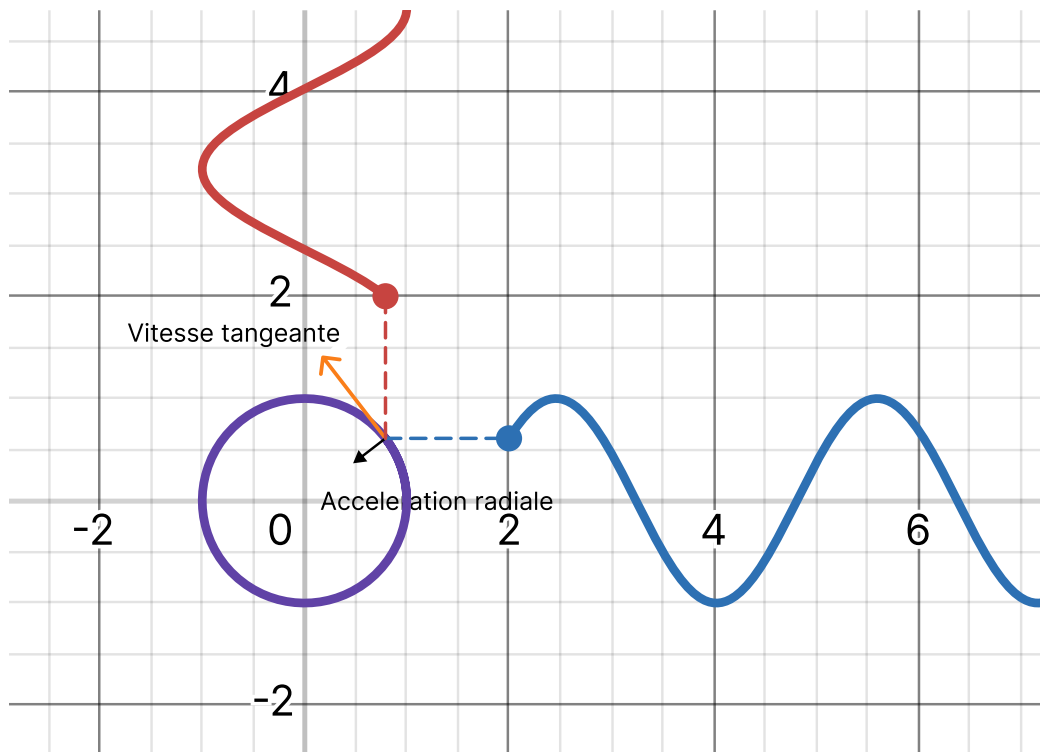


Fig. 7. – Le mouvement circulaire uniforme

3. Mouvement parabolique (projectile) :

- Position : $\vec{r}(t) = \begin{pmatrix} v_{0x} t \\ y_0 + v_{0y} t - \frac{1}{2} g t^2 \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} v_{0x} \\ v_{0y} - g t \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} 0 \\ -g \end{pmatrix}$

L'accélération est **constante** et dirigée vers le bas. Avec $g = 9.81 \text{ m/s}^2$, $\vec{a} = \begin{pmatrix} 0 \\ -9.81 \end{pmatrix} \text{ m/s}^2$. Seule la composante verticale de la vitesse change.

Session 3 : Les lois de Newton – La méthode physique

Objectifs de la session

- Réviser la cinématique vue précédemment.
- Introduire les trois lois de Newton comme fondement du moteur physique.
- Comprendre la démarche scientifique : modéliser, prédire, confronter à l'expérience.
- Préparer le lien entre forces et mouvement pour la simulation.

La démarche physique : modéliser l'univers

💡 Physique du Jeu : Comment programmer l'univers

Isaac Newton n'a jamais codé en JavaScript, mais sans lui, aucun jeu vidéo moderne n'existerait. Les lois de Newton sont des modèles mathématiques basés sur l'observation. On les utilise tant qu'aucune expérience ne les invalide.

1. La Première Loi : L'Inertie

La Loi

Un objet au repos reste au repos, et un objet en mouvement continue en ligne droite à vitesse constante, **sauf** si une force extérieure agit sur lui.

Ce que ça veut dire : Les objets résistent au changement. Pour modifier leur vitesse (accélérer, freiner, tourner), il faut appliquer une force.

Exemples :

- Bus qui freine : le corps continue d'avancer.
- **Space Invaders** : le personnage s'arrête instantanément quand on lâche la manette — physiquement faux, comme s'il y avait une friction infinie.
- **Asteroids** : le vaisseau continue en ligne droite sans friction — plus réaliste.

2. La Deuxième Loi : La Dynamique

💬 La Formule

$$\vec{F} = m \cdot \vec{a}$$

En pratique, on cherche l'accélération :

$$\vec{a} = \frac{\vec{F}}{m}$$

Ce que ça veut dire : L'accélération dépend de la force appliquée et de la masse de l'objet.

Exemples :

- Caddie de supermarché : vide, il part vite ; plein, il part lentement avec la même force.
- **Balancing FPS** : un Scout léger (50 kg) accélère à 10 m/s² avec 500 N, tandis qu'un Tank lourd (200 kg) n'accélère qu'à 2.5 m/s².

3. La Troisième Loi : Action-Réaction

La Loi

Pour toute action (force), il y a une réaction égale et opposée.

$$\vec{F}_{A \rightarrow B} = -\vec{F}_{B \rightarrow A}$$

Exemples :

- **Skateboard** : on pousse le sol vers l'arrière, le sol nous pousse vers l'avant.
- **Recul d'arme** : la balle part vers l'avant, l'arme recule vers l'arrière.
- **Rocket jump** : la roquette pousse le sol vers le bas, le joueur est propulsé vers le haut.
- **Collisions** : la boule blanche de billard transmet son impulsion à la boule rouge.

Résumé pour le développeur

Loi	Concept	Implémentation Code
1. Inertie	Les objets gardent leur vitesse.	Ne remettez pas velocity à 0 à chaque frame !
2. F = ma	Force -> Accélération.	<code>acc = forces / mass</code>
3. Action-Réaction	Les interactions sont doubles.	Collision : A repousse B, B repousse A.

Tableau 1. – Aide-mémoire des lois de Newton

Annexe : Pourquoi des objets de masses différentes tombent-ils avec la même vitesse ?

La force de pesanteur est le poids : $\vec{P} = m \cdot \vec{g}$.

La deuxième loi donne : $\vec{a} = \frac{\vec{F}}{m} = \frac{m \cdot \vec{g}}{m} = \vec{g}$.

La masse s'annule. L'accélération gravitationnelle est donc la même pour tous les objets.

Session 4 : Les forces de la nature

Objectifs de la session

- Identifier les forces de la nature agissant sur un objet dans un jeu.
- Comprendre le poids, la normale, le frottement sec et le frottement visqueux.
- Savoir quand et comment modéliser chaque force pour un gameplay réaliste.

Introduction

Dans le vide spatial, un objet lancé garde sa vitesse pour l'éternité (1ère loi de Newton). Sur Terre, tout finit par s'arrêter. Pourquoi ? À cause des interactions avec l'environnement, modélisées par les **forces de la nature**.

1. La Gravité et le Poids

La Gravité

La force d'attraction gravitationnelle attire les objets vers le centre de la Terre. Son accélération est notée $\vec{g}$.

$$\vec{P} = m \cdot \vec{g}$$

Application en jeu : La gravité est une force constante appliquée chaque frame à tous les objets dynamiques.

2. La Force Normale

La Force Normale (N)

C'est la force avec laquelle une surface repousse un objet qui appuie contre elle. Elle est perpendiculaire à la surface.

- Sur un sol plat horizontal : $N = mg$.
- Sur une pente : $N = mg \cos(\theta)$.

Application en jeu : La normale détermine si un objet est au sol et calcule la friction.

3. Le Frottement Sec (Modèle de Coulomb)

Frottement Sec

Frottement de contact entre deux solides. Il dépend du coefficient de friction μ et de la normale N .

$$\|\vec{f}\| = \mu \cdot \|\vec{N}\|$$

Statique vs Cinétique

- **Frottement statique (μ_s) :** l'objet est immobile. La force s'adapte jusqu'à un seuil critique $F_{\max} = \mu_s N$.

- **Frottement cinétique (μ_k)** : l'objet glisse. La force est constante $F_k = \mu_k N$.
- En général : $\mu_s > \mu_k$.

4. Le Frottement Visqueux (Fluide)

Frottement Visqueux

Résistance de l'air ou de l'eau. Il dépend de la vitesse et de la forme de l'objet.

- Vitesse faible : $\vec{f} = -k \cdot \vec{v}$.
- Vitesse élevée : $\vec{f} = -C \cdot \|v\|^2 \cdot \hat{v}$.

Conséquence : la vitesse terminale. Quand la gravité et le drag s'annulent, l'objet ne peut plus accélérer.

Exemple. Comparaison pour le Gameplay :

- **Frottement sec** : l'objet ralentit linéairement et s'arrête net.
- **Frottement fluide** : l'objet ralentit de moins en moins vite, approche asymptotique de 0.

Session 5 : Intégration numérique 101 - Méthode d'Euler

Objectifs de la session

- Comprendre pourquoi l'ordinateur discrétise le temps (frames, pas Δt).
- Apprendre la méthode d'Euler pour intégrer position et vitesse.
- Introduire la série de Taylor comme fondement des méthodes d'intégration.
- Identifier les limites d'Euler et quand l'utiliser malgré tout.

💡 Comment l'ordinateur prédit le futur

Dans nos simulations, le temps n'est pas continu. L'ordinateur calcule le monde image par image (souvent 60 FPS, soit $\Delta t \approx 0.016$ s). Les équations physiques continues (dérivées) doivent devenir des instructions discrètes (additions).

1. La Méthode d'Euler

L'Algorithme

Pour chaque pas de temps Δt :

1. Calculer l'accélération : $\vec{a}_n = \frac{\vec{F}}{m}$
2. Mettre à jour la vitesse : $\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \cdot \Delta t$
3. Mettre à jour la position : $\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \cdot \Delta t$

Intuition : On suppose que la vitesse est constante pendant tout le pas de temps.

2. D'où ça vient ? La dérivée discrète

La vitesse est la dérivée de la position :

$$\vec{v}(t) = \frac{d\vec{r}}{dt} \approx \frac{\vec{r}(t + \Delta t) - \vec{r}(t)}{\Delta t}$$

En isolant le terme futur :

$$\underbrace{\vec{r}(t + \Delta t)}_{\text{Futur}} \approx \underbrace{\vec{r}(t)}_{\text{Présent}} + \underbrace{\vec{v}(t) \cdot \Delta t}_{\text{Pas}}$$

3. La Série de Taylor

Approximation d'une fonction

La série de Taylor dit qu'une trajectoire peut être reconstruite en additionnant ses dérivées successives :

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t)\Delta t + \frac{1}{2}\vec{a}(t)\Delta t^2 + \frac{1}{6}\vec{j}(t)\Delta t^3 + \dots$$

Euler ne garde que le premier terme. C'est une méthode du **premier ordre**.

Pourquoi Euler est une approximation ?

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t + \underbrace{O(\Delta t^2)}_{\text{Erreur}}$$

4. Conséquences pratiques

- Euler ignore la courbure de la trajectoire (l'accélération) à l'intérieur du pas.
- Il tend à « dérapier » vers l'extérieur des virages ou à gagner de l'énergie.
- Malgré cela, il reste simple et rapide, et fonctionne quand l'accélération change (vent, ressorts, collisions).

Euler (Ordre 1)	Intégration exacte (Ordre 2)
$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t$ <i>Prend la pente au début et trace une ligne droite.</i>	$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t + \frac{1}{2} \vec{a} \Delta t^2$ <i>Prend en compte la courbure.</i>
Facile à coder. Rapide.	Exact pour la gravité constante.

Tableau 2. – Comparaison pour un pas de temps Δt

Session 6 : L'impulsion, les collisions

Objectifs de la session

- Comprendre l'impulsion comme changement de quantité de mouvement.
- Apprendre la loi de restitution et la vitesse relative.
- Calculer l'impulsion d'une collision entre deux objets.
- Implémenter un laboratoire de chocs 1D.

1. D'où vient l'Impulsion ?

Théorème de l'Impulsion

La force est la dérivée de la quantité de mouvement :

$$\vec{F} = \frac{d\vec{p}}{dt}$$

En intégrant sur un choc instantané ($\Delta t \approx 0$) :

$$\vec{J} = \Delta\vec{p} = m \cdot \Delta\vec{v}$$

Interprétation : L'impulsion est l'aire sous la courbe force-temps. Un choc bref et intense peut changer la vitesse instantanément.

2. La Loi de Restitution

Coefficient de restitution (e)

La vitesse à laquelle deux objets s'éloignent après un choc est proportionnelle à la vitesse à laquelle ils se rapprochaient avant le choc.

$$v_{\text{rel}}(\text{après}) = -e \cdot v_{\text{rel}}(\text{avant})$$

- $e = 1$: choc élastique (billes de billard).
- $e = 0$: choc mou (pâte à modeler).

3. Vitesse Relative et Normale

Vitesse relative scalaire

La vitesse de rapprochement projetée sur la normale de collision :

$$v_{\text{rel}} = (\vec{v}_A - \vec{v}_B) \cdot \vec{n}$$

- $v_{\text{rel}} < 0$: les objets se rapprochent.
- $v_{\text{rel}} > 0$: les objets s'éloignent.
- $v_{\text{rel}} = 0$: ils glissent l'un contre l'autre.

4. Calcul de l'Impulsion

🗨 Formule de l'impulsion

$$j = \underbrace{-(1+e)v_{\text{rel}}}_{\text{Vitesse à changer}} \cdot \underbrace{\left(\frac{m_A m_B}{m_A + m_B}\right)}_{\text{Masse réduite}}$$

Application aux vitesses :

$$v'_A = v_A + \left(\frac{j}{m_A}\right) \cdot \vec{n}$$

$$v'_B = v_B - \left(\frac{j}{m_B}\right) \cdot \vec{n}$$

5. TP : Laboratoire de Chocs 1D

💡 Objectifs du TP

Implémenter la réponse physique d'une collision entre deux boules de masses différentes dans un espace 1D sans gravité ni friction.

Missions

1. Créer deux boules : A ($m = 5$) lancée vers B ($m = 1$).
2. Calculer la normale $\vec{n}$.
3. Calculer la vitesse relative v_{rel} .
4. Si $v_{\text{rel}} > 0$, sortir (les objets s'éloignent).
5. Calculer l'impulsion j avec la masse réduite.
6. Appliquer le changement de vitesse aux deux boules.

Scénarios à observer

- **Pendule de Newton** : masses égales, $e = 1$ — A s'arrête, B repart.
- **Le Mur** : $m_A = 1$, $m_B = 100$ — A rebondit, B bouge à peine.
- **Bulldozer** : $m_A = 100$, $m_B = 1$ — A continue, B repart à environ $2v_A$.
- **Pâte à modeler** : $e = 0$ — les boules se collent.

Session 7 : Broad et Narrow phases – Part 1

Objectifs de la session

- Comprendre le budget temps d'une frame (16 ms).
- Exploquer pourquoi la détection naïve de collision est en $O(N^2)$.
- Présenter la Broad Phase et ses structures de partitionnement.
- Comparer Grille Uniforme, Quadtree/Octree et Sweep-and-Prune.

1. Le Contexte : Le Budget Temps (16 ms)

Le budget temps d'une frame

À 60 FPS, on dispose de **16.6 millisecondes** pour tout calculer :

- Rendu : 8 ms
- IA / Gameplay : 4 ms
- Audio / Réseau : 2 ms
- **Physique : 2 ms**

Si la détection de collision prend 10 ms, le jeu saccade. L'optimisation est vitale.

2. Le Problème : La Malédiction $O(N^2)$

Détection naïve

Avec N objets, chaque objet teste tous les autres. Le nombre de paires est :

$$C \approx \frac{N(N-1)}{2} \approx O(N^2)$$

- 10 objets → 45 tests
- 1 000 objets → 499 500 tests
- 10 000 objets → 50 000 000 tests

Solution : réduire le nombre de paires avant les tests coûteux. C'est le rôle de la **Broad Phase**.

3. La Broad Phase : Structures de Partitionnement

Principe

On entoure chaque objet complexe par une AABB (Axis Aligned Bounding Box). La Broad Phase répond vite à la question : « **Qui sont mes voisins ?** »

A. Grille Uniforme (Spatial Hashing)

- On découpe le monde en cases de taille fixe.
- On ne teste un objet que contre ceux dans la même case ou les cases adjacentes.
- **Avantage** : accès $O(1)$, rapide, facile à coder.
- **Inconvénient** : mauvais si le monde est infini ou si les tailles varient beaucoup.

B. Arbres Spatiaux (Quadtree / Octree)

- Découpage récursif. Si une case contient trop d'objets, on la coupe en 4 (2D) ou 8 (3D).
- **Avantage** : s'adapte à la densité.

- **Inconvénient** : coûteux à mettre à jour si les objets bougent beaucoup.

C. Sweep and Prune (SAP)

- On projette les débuts/fins des AABB sur les axes X, Y, Z et on trie les listes.
- Si les intervalles sur X ne se chevauchent pas, inutile de tester Y et Z.
- Exploite la **cohérence temporelle** : la liste reste presque triée d'une frame à l'autre.

Session 8 : Les ressorts - Méthode de Verlet

Objectifs de la session

- Comprendre la loi de Hooke et les forces de rappel.
- Stabiliser les ressorts avec le damping.
- Découvrir l'intégration de Verlet pour les systèmes de particules.
- Apprendre à résoudre les contraintes de distance (PBD).

1. Les Ressorts : La Loi de Hooke

La Loi de Hooke

Un ressort est une force qui ramène un objet à sa position d'équilibre.

$$\vec{F} = -k \cdot (L - L_0) \cdot \hat{u}$$

- k : raideur du ressort.
- L : longueur actuelle.
- L_0 : longueur au repos.
- $\hat{u}$: direction unitaire.

Problème de stabilité : Avec Euler et un k élevé, le système « explose » et accumule de l'énergie.

2. Le Damping (Amortissement)

Force d'amortissement

On ajoute une force opposée à la vitesse pour dissiper l'énergie.

$$\vec{F}_{\text{damping}} = -b \cdot \vec{v}$$

Résultat : Le ressort finit par se stabiliser au lieu d'osciller indéfiniment.

3. L'Intégration de Verlet

Verlet

Contrairement à Euler, Verlet ne stocke pas explicitement la vitesse. Elle est déduite de la différence entre la position actuelle et la position précédente.

$$x_{n+1} = x_n + (x_n - x_{n-1}) + a \cdot dt^2$$

Avantages :

- Stable pour les ressorts et les oscillations.
- Si on déplace une particule manuellement (contrainte), sa vitesse s'ajuste automatiquement.

Utilisations typiques : cordes, tissus, chevelures, tas de billes.

4. Contraintes de Verlet (PBD)

Position Based Dynamics

Au lieu de calculer des forces complexes, on corrige directement les positions des particules pour respecter des contraintes (distance, angle, etc.).

Algorithme :

1. Mettre à jour toutes les particules avec Verlet (gravité, etc.).
2. Pour chaque contrainte de distance, calculer l'écart entre la distance actuelle et la distance souhaitée.
3. Déplacer les deux particules pour corriger la moitié de l'écart chacune.
4. Répéter plusieurs fois (itérations) pour converger.

Avantage : pas de calcul de forces, manipulation directe des positions. C'est la base des moteurs PBD.

Session 9 : TP - Particules

Objectifs du TP

- Implémenter un système de particules simple.
- Utiliser Verlet ou Euler pour mettre à jour les particules.
- Créer des contraintes de distance pour former des structures (tissu, corde).
- Observer les cycles de vie naissance / update / mort.

Contexte

Un système de particules simule un grand nombre de petits objets : pluie, feu, fumée, étincelles. C'est le moment d'appliquer concrètement l'intégration vue précédemment.

1. Structure d'une Particule

Propriétés d'une particule

- Position, vitesse, accélération
- Durée de vie (lifetime)
- Couleur, taille
- Masse (optionnelle)

2. L'Émetteur

Emitter

- Génère des particules à un taux donné (ex: 1000/s).
- Définit position, direction, vitesse initiale.
- Peut être un point, un cercle, une sphère ou un cône.

3. Le Cycle de Vie

Cycle de vie

1. **Naissance** : création avec paramètres initiaux.
2. **Update** : application des forces (gravité, drag), mise à jour de la position.
3. **Rendu** : dessin sous forme de billboard (quad face caméra).
4. **Mort** : désactivation et recyclage quand lifetime ≤ 0 .

4. Interactions Physiques

- **Gravité** : $\vec{a} = \vec{g}$.
- **Drag** : $\vec{F} = -k \cdot \vec{v}$.
- **Collisions simplifiées** : rebond sur un plan, par exemple.

```
if (particle.position.y < 0) {
    particle.position.y = 0;
    particle.velocity.y *= -restitution; // rebond mou
}
```

5. Contraintes et Structures

💡 Objectif avancé

Relier des particules par des contraintes de distance pour créer un tissu ou une corde. Utiliser Verlet pour la stabilité et résoudre les contraintes en plusieurs itérations.

6. Défis

- Créer un émetteur de pluie.
- Créer un feu de camp avec des particules qui montent et s'estompent.
- Créer un drapeau (tissu) avec des contraintes de distance.

Session 10 : Introduction à la physique des rotations

Objectifs de la session

- Passer de la particule au corps rigide.
- Comprendre le couple, le moment d’inertie et la vitesse angulaire.
- Relier les équations linéaires et angulaires.
- Implémenter un corps rigide qui tourne sous l’effet d’une force.

💡 L'objectif du jour

Un corps rigide ne fait pas que se déplacer, il **pivote** autour de son centre de masse. Cette session introduit les outils pour modéliser la rotation.

1. Dictionnaire Linéaire vs Angulaire

Concept	Linéaire	Angulaire
Position	$\vec{r}$ (m)	Angle θ (rad)
Vitesse	$\vec{v}$ (m/s)	Vitesse angulaire ω (rad/s)
Résistance	Masse m	Moment d’inertie I
Cause	Force $\vec{F}$	Couple τ
Loi	$\vec{F} = m\vec{a}$	$\tau = I\alpha$

Tableau 3. – Analogie entre mouvement linéaire et rotation

2. Le Couple (Torque)

💡 Le Couple

Le couple dépend de **où** on pousse. En 3D, il est donné par un produit vectoriel :

$$\vec{\tau} = \vec{r} \times \vec{F}$$

En 2D, il se simplifie :

$$\tau = r_x F_y - r_y F_x$$

où $\vec{r}$ est le bras de levier depuis le centre de masse.

3. Le Moment d’Inertie

Moment d’inertie

Le moment d’inertie I représente la résistance d’un objet à la rotation. Il dépend de la répartition de la masse.

$$I = \sum_i m_i r_i^2$$

Cas courants :

- Anneau mince : $I = mR^2$
- Rectangle autour de son centre : $I = \frac{1}{12}m(w^2 + h^2)$

4. Vitesse d'un Point sur le Corps**Vitesse d'un point P**

Si un corps se déplace à $\vec{v}_G$ et tourne à ω , la vitesse d'un point P à la distance $\vec{r}$ du centre est :

$$\vec{v}_P = \vec{v}_G + (\omega \times \vec{r})$$

En 2D :

$$v_{P.x} = v_{G.x} - \omega \cdot r_y$$

$$v_{P.y} = v_{G.y} + \omega \cdot r_x$$

5. TP : Spinning Box**Objectif**

Implémenter un rectangle qui tourne quand on clique dessus avec la souris.

Étape 1 : Classe Rigidbody

Ajouter les propriétés :

- angle
- angularVelocity
- torque
- inertia (calculée une fois au démarrage)

Étape 2 : ApplyForce(force, worldPoint)

1. Calculer le bras de levier $\vec{r} = \text{worldPoint} - \text{center}$.
2. Ajouter la force linéaire : $\vec{F}_{\text{total}} += \vec{F}$.
3. Ajouter au torque : $\tau += r_x F_y - r_y F_x$.

Étape 3 : Intégrateur

1. $a = \frac{F}{m}$
2. $\alpha = \frac{\tau}{I}$
3. $v += a \cdot dt$
4. $\omega += \alpha \cdot dt$
5. $p += v \cdot dt$
6. $\theta += \omega \cdot dt$
7. Remettre F et tau à zéro.

Session 11 : Broad et Narrow phases – Part 2

Objectifs de la session

- Comprendre le rôle de la Narrow Phase.
- Apprendre SAT et GJK pour la détection précise de collision.
- Voir comment générer un contact et résoudre la collision.

Rappel

La Broad Phase a filtré les paires candidates. La Narrow Phase répond à la question : « **Ces deux objets se touchent-ils vraiment ?** »

1. La Narrow Phase : Méthodes de Résolution

Objectif

On a maintenant des paires proches. Il faut déterminer si elles se touchent vraiment, puis générer un contact (point, normale, profondeur).

2. SAT (Separating Axis Theorem)

Principe de SAT

Si on peut trouver un axe sur lequel les projections de deux objets ne se chevauchent pas, alors ils ne se touchent pas.

On teste les axes normaux des faces et les produits vectoriels des arêtes. Si un seul axe sépare les objets, il n'y a pas collision.

Utilisation : Boîtes vs boîtes, polyèdres convexes simples.

3. GJK (Gilbert-Johnson-Keerthi)

Principe de GJK

On travaille dans l'espace de Minkowski ($A - B$). On cherche si l'origine $(0, 0, 0)$ est contenue dans cette forme en construisant un simplex (tétraèdre en 3D).

Fonction de Support

GJK pose la question : « **Quel est ton point le plus extrême dans la direction $\vec{d}$?** »

$$S_{A-B}(\vec{d}) = S_A(\vec{d}) - S_B(-\vec{d})$$

- Sphère : Centre + Rayon $\cdot \vec{d}$
- Cube : coin correspondant au signe de $\vec{d}$

Utilisation : Formes convexes quelconques (capsules, cônes, meshes convexes).

4. Génération du Contact et Résolution

Contact

Une fois la collision détectée, on a besoin de :

- Point de contact.
- Normale de collision.
- Profondeur de pénétration.

Résumé du Pipeline

1. **Update Position** : $P = P + V \cdot dt$
2. **Broad Phase** : voisinage -> paires candidates.
3. **Narrow Phase** : AABB, SAT/GJK -> collision OUI/NON.
4. **Contact Generation** : point, normale, profondeur.
5. **Resolution** : impulsions pour séparer les objets.

Session 12 : TP - Les planètes - Runge-Kutta

Objectifs du TP

- Simuler un système solaire avec la gravité universelle.
- Comprendre pourquoi Euler « dérape » sur les orbites.
- Implémenter Runge-Kutta 4 pour une intégration plus stable.
- Trouver la bonne vitesse initiale pour une orbite circulaire.

1. La Gravité Universelle

Loi de gravitation universelle

Newton a découvert que tous les objets s'attirent mutuellement.

$$\vec{F} = G \cdot \frac{m_1 m_2}{r^2} \cdot \hat{u}$$

- G : constante gravitationnelle (6.674×10^{-11}).
- r : distance entre les centres.
- $\hat{u}$: direction de l'attraction.

Pourquoi on ne ressent pas l'attraction entre deux personnes ? Parce que G est minuscule. Il faut une masse planétaire.

2. Le Système Solaire

Objectif du TP

1. Créer un soleil massif au centre.
2. Ajouter des planètes avec des vitesses initiales perpendiculaires au soleil.
3. Appliquer la gravité universelle entre chaque paire.
4. Observer les orbites.

Défi : trouver la vitesse initiale pour une orbite circulaire parfaite.

$$v = \sqrt{G \frac{M}{r}}$$

3. Pourquoi Euler ne suffit pas

Euler suppose que la vitesse est constante sur tout le pas de temps. Sur une orbite, la force change constamment, donc Euler accumule de l'erreur et les planètes « dérapent » ou s'échappent.

4. Runge-Kutta 4 (RK4)

Algorithme RK4

RK4 « sonde » le terrain à 4 endroits du pas de temps pour obtenir une pente moyenne.

1. $k_1 = \text{eval}(\text{état})$
2. $k_2 = \text{eval}(\text{état} + k_1 * h/2)$
3. $k_3 = \text{eval}(\text{état} + k_2 * h/2)$

$$4. \quad k_4 = \text{eval}(\text{état} + k_3 * h)$$

$$\text{Nouvel état} = \text{état} + \frac{k_1 + 2k_2 + 2k_3 + k_4}{6} * h$$

Structure d'État

On regroupe position et vitesse dans un vecteur d'état Y .

$$Y = \begin{pmatrix} x \\ v \end{pmatrix}$$

$$\dot{Y} = \begin{pmatrix} v \\ a \end{pmatrix}$$

5. Missions

- Implémenter la gravité universelle entre N objets.
- Comparer Euler vs RK4 sur une orbite simple.
- Observer la stabilité des orbites sur plusieurs révolutions.
- Ajouter plusieurs planètes et voir leurs interactions.

Session 13 : Les moteurs physiques commerciaux

Objectifs de la session

- Comprendre pourquoi on utilise des moteurs physiques existants en production.
- Découvrir les moteurs majeurs et leurs cas d'usage.
- Introduire le déterminisme et l'architecture ECS.

1. Pourquoi utiliser un moteur physique ?

Avantages des moteurs existants

- **Complexité** : gestion des contacts multiples, stabilité, optimisation.
- **Temps** : des décennies de R&D.
- **Performance** : optimisé pour des milliers d'objets.

2. Concepts avancés

Rigid Body vs Soft Body

- **Rigid Body** : objet indéformable (boîte, sphère).
- **Soft Body** : objet déformable (tissu, gelée), beaucoup plus complexe.

Continuous Collision Detection (CCD)

Problème : un objet rapide peut traverser un mur entre deux frames (tunneling). **Solution** : on raycaste la trajectoire entre t et $t + \Delta t$ au lieu de tester seulement les positions.

Sleeping

Si un objet est immobile, on arrête de le simuler. Il « dort » et se réveille si un autre objet le touche.

3. Comparatif des moteurs

Moteur	Langage	Usage
PhysX	C++	AAA Games (Unity, Unreal default)
Havok	C++	AAA Games (Half-Life 2, Zelda TotK)
Jolt Physics	C++	Modern, open-source, Horizon Forbidden West
Box2D	C++	2D Games (Angry Birds, Terraria)
Cannon.js	JavaScript	Three.js (simple, legacy)
Ammo.js	JavaScript (WASM)	Three.js (Bullet port)
Rapier	Rust/WASM	Modern, fast, Three.js recommended

Tableau 4. – Comparatif des moteurs physiques

4. Déterminisme

Déterminisme

Un moteur est déterministe si les mêmes inputs produisent toujours les mêmes outputs.

Importance : multijoueur, replay, replays de gameplay.

Pièges : ordre des opérations, précision flottante selon le CPU.

5. Architecture ECS

Entity-Component-System

- **Entity :** un simple ID (ex: entity_42).
- **Component :** données pures (ex: Position, RigidBody, Mesh).
- **System :** logique qui opère sur les composants (ex: PhysicsSystem, RenderSystem).

Avantage : flexibilité, on ajoute/enlève des composants à la volée.

Session 14 : Corps rigides – Boite de collision

Objectifs de la session

- Utiliser un moteur physique (Rapier.js) pour créer des corps rigides.
- Comprendre les types de Rigidbody.
- Découvrir les Colliders et leurs formes.
- Synchroniser le moteur physique avec Three.js.

🔗 Introduction

Maintenant que nous comprenons les concepts, nous allons utiliser un vrai moteur : **Rapier.js**. C'est un moteur 3D écrit en Rust, compilé en WebAssembly, et recommandé pour Three.js.

1. Les RigidBodies

Rigidbody

Un Rigidbody dans Rapier est l'objet principal. Il a :

- position, rotation, vitesse linéaire et angulaire.
- masse.
- un type : Dynamic, Fixed, Kinematic.

Types :

- **Dynamic** : réagit aux forces et collisions.
- **Fixed** : immobile, les autres rebondissent contre.
- **Kinematic** : contrôlé par le code, pousse les Dynamic.

Créer un Rigidbody dans Rapier.

```
import RAPIER from '@dimforge/rapier3d';

const world = new RAPIER.World({ x: 0, y: -9.81, z: 0 });

const rigidBodyDesc = RAPIER.RigidBodyDesc.dynamic()
  .setTranslation(0, 5, 0);
const rigidBody = world.createRigidBody(rigidBodyDesc);

const colliderDesc = RAPIER.ColliderDesc.ball(1.0);
const collider = world.createCollider(colliderDesc, rigidBody);
```

2. Les Colliders

Collider

Le Collider définit la forme de l'objet pour la détection. Un Rigidbody peut avoir plusieurs Colliders.

Formes disponibles :

- ball(radius) : sphère.
- cuboid(hx, hy, hz) : boîte.
- capsule(halfHeight, radius) : capsule.

- `cylinder(halfHeight, radius)` : cylindre.
- `convexHull(points)` : forme convexe personnalisée.
- `trimesh(vertices, indices)` : mesh quelconque (statique).

3. Boucle de simulation

Synchronisation Three.js.

```
function animate() {  
    requestAnimationFrame(animate);  
  
    world.step();  
  
    for (const { mesh, rigidBody } of objects) {  
        const position = rigidBody.translation();  
        const rotation = rigidBody.rotation();  
        mesh.position.set(position.x, position.y, position.z);  
        mesh.quaternion.set(rotation.x, rotation.y, rotation.z, rotation.w);  
    }  
  
    renderer.render(scene, camera);  
}
```


Session 15 : Joints, Moteurs

Objectifs de la session

- Comprendre les types de joints et leurs cas d'usage.
- Savoir configurer des moteurs et des limites sur les joints.
- Découvrir les raycasts comme outil de requête dans le moteur physique.

1. Les Joints

Joints (Articulations)

Les joints relient deux RigidBody, limitant leur mouvement relatif.

Types :

- **Fixed Joint** : les deux corps sont rigidement liés (soudure).
- **Revolute Joint** : rotation autour d'un axe (charnière).
- **Prismatic Joint** : translation le long d'un axe (tiroir).
- **Spherical Joint** : rotation libre (épaule, rotule).

Configuration : chaque joint nécessite des **ancres** (anchors) locales sur chaque corps.

Créer une charnière dans Rapier.

```
const jointDesc = RAPIER.JointData.revolute(
  { x: 0, y: 1, z: 0 }, // ancre sur A
  { x: 0, y: -1, z: 0 }, // ancre sur B
  { x: 0, y: 1, z: 0 } // axe de rotation
);
const joint = world.createJoint(jointDesc, bodyA, bodyB);
```

2. Les Moteurs et Limites

Moteurs

On peut ajouter des moteurs aux joints pour les faire bouger activement.

```
joint.configureMotorVelocity(targetVelocity, stiffness, damping);
```

Limites

On ajoute des limites (min/max angle) pour empêcher une rotation excessive.

3. Les Raycasts

Raycast

Un raycast projette un rayon invisible et retourne le premier objet touché.

Utilisations : détection du sol, visée FPS, interaction, capteurs de véhicule.

Raycast dans Rapier.

```
const ray = new RAPIER.Ray(  
  { x: 0, y: 10, z: 0 },  
  { x: 0, y: -1, z: 0 }  
);  
const hit = world.castRay(ray, 20.0, true);  
if (hit) {  
  console.log("Distance:", hit.timeOfImpact);  
}
```

4. TP : Machine de Siège

Objectif

Construire une catapulte fonctionnelle :

1. Créer une base fixe.
2. Créer un bras avec un revolute joint.
3. Ajouter un moteur au joint.
4. Lancer un projectile.

Session 16 : Les contraintes – et l'IK

Objectifs de la session

- Comparer PBD et Impulse Based pour résoudre les contraintes.
- Comprendre l'algorithme PGS et les techniques de stabilité.
- Passer de la cinématique directe (FK) à l'inverse (IK).
- Découvrir CCD, FABRIK et leurs applications.

1. PBD vs Impulse Based

Deux approches pour résoudre les contraintes

A. Position Based Dynamics (PBD)

- Corrige directement les positions.
- Avantage : simple, stable visuellement.
- Inconvénient : la physique n'est pas « vraie » (vitesses incorrectes).
- Utilisé par : Verlet, PhysX (Soft Body), BeamNG.

B. Impulse Based

- Calcule les impulsions nécessaires pour respecter les contraintes.
- Avantage : physiquement correct, conserve la quantité de mouvement.
- Inconvénient : plus complexe, nécessite plusieurs itérations.
- Utilisé par : Box2D, Rapier, Havok, PhysX (Rigid Body).

2. Le Concept de Contrainte

Contrainte

Une contrainte est une équation qui doit être satisfaite à chaque frame.

Exemple : distance L entre deux particules A et B.

$$C = \|\overrightarrow{AB}\| - L = 0$$

Si $C \neq 0$, on applique une correction.

3. Algorithme PGS (Projected Gauss-Seidel)

PGS

Principe :

1. Pour chaque contact, calculer l'impulsion nécessaire.
2. L'appliquer.
3. Les vitesses ont changé, donc on recommence (itération).
4. Après N itérations, le système converge.

Typique : 4 à 8 itérations.

4. Techniques de Stabilité

Stabilité

Warm Starting : réutiliser les impulsions de la frame précédente pour converger plus vite.

Sleeping : les objets immobiles ne sont pas simulés.

Position Correction (Baumgarte) : corriger légèrement la position pour réduire la pénétration.

$$\text{correction} = \text{penetration} \cdot \text{facteur} \cdot \vec{n}$$

5. Forward vs Inverse Kinematics

FK vs IK

Forward Kinematics (FK) : on donne les angles et on calcule la position de la main.

$$\vec{p} = f(\theta_1, \theta_2, \dots)$$

Inverse Kinematics (IK) : on donne la position de la main et on calcule les angles.

$$\theta_1, \theta_2, \dots = f^{-1}(\vec{p})$$

Difficultés : plusieurs solutions possibles, parfois aucune solution, pas de solution analytique simple.

6. CCD (Cyclic Coordinate Descent)

CCD

On ajuste les articulations une par une, de l'extrémité vers la base.

1. Prendre la dernière articulation.
2. Calculer l'angle entre (articulation -> cible) et (articulation -> main).
3. Tourner l'articulation.
4. Passer à l'articulation précédente.
5. Répéter.

Avantage : simple, converge vite. **Inconvénient** : mouvements peu naturels.

7. FABRIK

FABRIK

On manipule directement les positions des articulations.

1. **Forward** : de la main vers l'épaule, en respectant les distances.
2. **Backward** : de l'épaule vers la main, en respectant les distances.
3. Répéter jusqu'à convergence.

Avantage : mouvements naturels, rapide. **Inconvénient** : ne gère pas les limites d'angles directement.

8. Applications de l'IK

- **Foot placement** : adapter les pieds au terrain.
- **Active ragdoll** : maintenir une pose via IK.
- **Hand grab** : attraper un objet précis.

Session 17 : TP - Véhicules

Objectifs du TP

- Comprendre le modèle « Raycast Vehicle ».
- Implémenter une suspension simple.
- Gérer la tenue de route (grip, slip angle).
- Créer un véhicule drivable avec Rapier.

1. Le Modèle Raycast Vehicle

Raycast Vehicle

Pour les jeux arcade, on ne simule pas les vraies roues. On utilise un raycast depuis le châssis vers le sol pour chaque roue.

Anatomie :

- Un châssis (RigidBody dynamique).
- Des roues invisibles : raycasts vers le bas.

Avantages : plus stable, plus performant, pas besoin de RigidBody pour les roues.

2. La Suspension

Suspension

Chaque roue simule un ressort.

1. Raycast depuis la roue vers le bas.
2. Si on touche le sol à distance d :
 - Compression : $c = 1 - \frac{d}{\text{maxSuspensionLength}}$
 - Force : $F = k \cdot c - \text{damping} \cdot v_{\text{suspension}}$
3. Appliquer la force au châssis.

3. La Tenue de Route

Grip et Slip Angle

- **Force longitudinale** : accélération / freinage.
- **Force latérale** : virage.
- **Slip angle** : angle entre la direction du pneu et la direction réelle du mouvement.
 - Petit angle : la voiture tourne normalement.
 - Grand angle : la voiture dérape (drift).

4. Missions du TP

1. Créer un châssis (boîte dynamique).
2. Ajouter 4 raycasts (roues).
3. Implémenter la suspension (ressort + damping).
4. Ajouter l'accélération (force longitudinale).
5. Ajouter la direction (force latérale sur les roues avant).

6. Tester sur un terrain off-road.

Défi

Trouver les bons paramètres de suspension pour que la voiture ne bascule pas dans les virages.

Session 18 : Personnages

Objectifs de la session

- Comparer Dynamic Character Controller (DCC) et Kinematic Character Controller (KCC).
- Implémenter Move and Slide.
- Gérer la détection du sol, le Coyote Time et le Jump Buffering.
- Créer un contrôleur FPS avec Rapier.

1. DCC vs KCC

Dynamic Character Controller (DCC)

Le personnage est un Rigidbody standard.

- **Avantages** : interagit naturellement avec le monde (poussé, tombe, etc.).
- **Inconvénients** : difficile à contrôler, peut glisser, être poussé.

Kinematic Character Controller (KCC)

Le personnage est un Rigidbody kinematic. On contrôle sa position manuellement.

- **Avantages** : contrôle total, game feel précis.
- **Inconvénients** : il faut coder les collisions, la gravité, les pentes.

La norme dans l'industrie : KCC pour les jeux AAA.

2. Move and Slide

Algorithme

1. Calculer le mouvement souhaité (input + gravité + vitesse).
2. Tenter de déplacer le personnage.
3. Si on touche un mur, glisser le long du mur.

Projection et Rejection

Soit $\vec{v}$ la vitesse et $\vec{n}$ la normale du mur.

- Projection (à annuler) : $\vec{v}_{\text{proj}} = (\vec{v} \cdot \vec{n})\vec{n}$
- Rejection (à conserver) : $\vec{v}_{\text{rej}} = \vec{v} - \vec{v}_{\text{proj}}$
- Nouvelle vitesse : $\vec{v}' = \vec{v}_{\text{rej}}$

3. Corner Trap

Quand on glisse le long de deux murs (un coin), le personnage peut se bloquer. Solution : traiter une collision à la fois, trier par profondeur, résoudre la plus profonde, puis re-tenter le mouvement.

4. Détection du Sol

Méthodes

- **Raycast simple** : un rayon depuis le centre. Simple mais peut rater les bords.

- **Shapecast** : on projette la capsule entière. Plus précis.
- **Multi-raycast** : plusieurs rayons sous les pieds.

Coyote Time et Jump Buffering

Coyote Time : après avoir quitté une plateforme, le joueur peut encore sauter pendant 0.1s.

Jump Buffering : si le joueur appuie sur saut juste avant d'atterrir, le saut s'exécute dès l'atterrissage.

5. TP : FPS Controller avec Rapier

1. Créer un Rigidbody kinematic (capsule).
2. Lire les inputs (WASD + souris).
3. Calculer la vitesse souhaitée.
4. Utiliser `world.castRay` pour détecter le sol.
5. Implémenter le Move and Slide.
6. Ajouter le Coyote Time et le Jump Buffering.

Session 19 : Physique, GPU et IA

Objectifs de la session

- Découvrir comment l'IA combine physique pour générer du mouvement.
- Comprendre Reinforcement Learning et Neuroevolution.
- Voir les capteurs physiques comme inputs d'un réseau de neurones.
- Introduire l'optimisation des particules avec le GPU.

1. Physique et IA : Mouvement Organique

Pourquoi mélanger IA et physique ?

Les animations pré-enregistrées sont rigides. La physique + l'IA permet de générer du mouvement en temps réel, adapté au terrain.

2. Reinforcement Learning (RL)

La boucle RL

- **Agent** : le personnage.
- **Environment** : le monde physique (Rapier).
- **State** : ce que l'agent voit (capteurs, positions, vitesses).
- **Action** : forces ou torques appliqués.
- **Reward** : note de l'action.

Exemple : apprendre à marcher. Reward = distance parcourue. Pénalité = tomber.

3. Neuroevolution (Genetic Algorithms)

Principe

1. Créer une population d'agents avec des réseaux de neurones aléatoires.
2. Les faire évoluer dans le monde.
3. Évaluer le fitness (distance, score).
4. Sélectionner les meilleurs.
5. Crossover et mutation.
6. Nouvelle génération.

4. Capteurs Physiques comme Inputs

Inputs du réseau

- **Raycasts** : N rayons vers l'avant, distance au premier obstacle.
- **Vitesse** : vitesse actuelle.
- **Orientation** : angle par rapport à la cible.

Exemple : 5 raycasts + vitesse + orientation = 7 inputs. 8 neurones cachés. 2 outputs (accélération + direction).

5. Optimisation GPU pour les Particules

Pourquoi le GPU ?

- Les particules sont indépendantes, idéales pour le parallélisme.
- Le GPU possède des milliers de cœurs.
- Les données restent sur le GPU (pas de transfert CPU/GPU).

Compute Shaders

- Programme qui s'exécute sur le GPU mais n'est pas un shader de rendu.
- Les particules sont stockées dans un SSBO (Shader Storage Buffer Object).
- Le compute shader met à jour positions, vitesses, lifetime.

Pipeline : CPU envoie les paramètres -> Compute Shader update -> Vertex/Fragment Shaders rendent.

6. TP : Voiture Autonome

Objectif

Créer une voiture qui apprend à naviguer un circuit :

1. Créer un circuit (murs, checkpoints).
2. Créer N voitures avec un Raycast Vehicle.
3. Chaque voiture a un réseau de neurones simple.
4. Inputs : 5 raycasts + vitesse.
5. Outputs : accélération + direction.
6. Fitness : distance parcourue avant de crasher.
7. Implémenter sélection, crossover, mutation.

Session 20 : Résumé - Quiz final

Objectifs de la session

- Réviser les concepts clés du cours.
- Faire le point sur les compétences acquises.
- Passer un quiz final pour valider les apprentissages.

Session présentielle

Révisions et Quiz (20 points).

Concepts clés à retenir

Fondamentaux

- Les trois lois de Newton.
- Les forces de la nature (gravity, normale, friction, drag).
- L'intégration numérique (Euler, Verlet, RK4).
- L'impulsion et les collisions.

Détection de collision

- Broad Phase (Grille, Quadtree, SAP).
- Narrow Phase (SAT, GJK).
- Génération de contacts et résolution.

Moteurs et systèmes avancés

- RigidBodies, Colliders, Joints, Moteurs.
- Contraintes et IK (CCD, FABRIK).
- Particules, GPU, personnages, véhicules, IA.

Quiz final

Format

Le quiz porte sur l'ensemble des sessions. Il mélange questions théoriques et petits problèmes à résoudre.